Senior Mobile Engineer

Objective

Build a native mobile application that allows a user to search for a city and see a ranked list of activities suitable for that location over the next 7 days, based on weather forecast data.

Use the Open-Meteo APIs:

- Geocoding API for city search
- Forecast API for weather data

The activities to rank are:

- Skiing
- Surfing
- Outdoor sightseeing
- Indoor sightseeing

You may complete the test in either:

- iOS using Swift
- Android using Kotlin

No backend is required.

Expected time: **3-4 hours**.

We care more about architecture, maintainability, correctness, testing, and communication than visual polish or feature volume.

You may choose which weather fields to use, but your README should explain your assumptions.

We are looking for:

- Clean native mobile architecture.
- Clear separation between UI, domain, API, and state management.
- Platform-appropriate implementation.
- Explicit state modeling.
- Testable ViewModels / presentation logic.
- Mockable API/data layer.

- Robust error handling.
- Meaningful unit tests.
- Clear README explaining decisions and trade-offs. Suitable architecture approaches may include:

iOS

- SwiftUI
- MVVM, Clean Architecture, or another justified structure.
- Async/await, Combine, or another justified async approach.
- Protocol-based dependency injection.
- XCTest. **Android**

- Jetpack Compose.
- MVVM, Clean Architecture, or another justified structure.
- Coroutines / Flow.
- Repository pattern.
- Hilt, Koin, or manual dependency injection.
- JUnit, MockK, Turbine, Robolectric, or equivalent tools where useful.

Deliverables

Submit a GitHub repository containing:

1. The native mobile application codebase.
2. A README covering:
    a. Project overview.
    b. Platform and tooling choices.
    c. Architecture and technical decisions.
    d. How to build and run the app.
    e. How to run tests & testing strategy.
    f. API usage notes.
    g. Activity recommendation logic.
    h. Assumptions made.
    i. Trade-offs and omissions.
    j. Production-readiness notes.
    k. Cross-platform delivery notes.
    l. AI usage disclosure, if applicable. AI usage is allowed. Poor verification is not.
3. Tests for the most important logic and state handling.

The app should build and run locally.

Bonus Points

The following are useful but not required:

- Snapshot tests.
- Full UI test coverage.
- Offline cache implementation.
- Pull-to-refresh.
- Dark mode polish.
- Advanced UI polish or animation.

Do not prioritize visual polish over architecture, correctness, and testability.